

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 8: RISC-V Part 3

Instructors: Justin Yokota

Agenda

- RISC-V Functions (Basic)
- RISC-V Memory Operations
- RISC-V Functions (Full)

Agenda

- **RISC-V Functions (Basic)**
- RISC-V Memory Operations
- RISC-V Functions (Full)

Hailstone: Where we left off

```
# Sets a0 to starting value of Hailstone Sequence
li a0 27

Loop:
mv s0 a0 # Save value of a0 in s0 so we don't lose it

# Print a0, followed by a newline character
add a1 a0 x0
li a0 1
ecall
li a0 11
li a1 10
ecall

mv a0 s0 # Restore a0 from s0 after print

# If a0 == 1, we're done; exit program
li t0 1
beq t0 a0 Done
```

```
# Check if a0 is even or odd
andi t1 a0 1
beq t1 x0 Even

# If odd,  $a0 = 3a0 + 1$ 
Odd:
slli t0 a0 1
add a0 a0 t0
addi a0 a0 1
j Finally # Skip past Even code

# If even,  $a0 = a0 / 2$ 
Even:
srli a0 a0 1

# Jump back to start of loop
Finally:
j Loop
Done:
# Exit program
li a0 10
li a1 0
ecall
```

Hailstone: Where we left off

```
# Sets a0 to starting value of Hailstone Sequence
li a0 27

Loop:
mv s0 a0 # Save value of a0 in s0 so we don't lose it

# Print a0, followed by a newline character
add a1 a0 x0
li a0 1
ecall
li a0 11
li a1 10
ecall

mv a0 s0 # Restore a0 from s0 after print

# If a0 == 1, we're done; exit program
li t0 1
beq t0 a0 Done
```

Print "function" is
inlined; can we
move it out?

```
# Check if a0 is even or odd
andi t1 a0 1
beq t1 x0 Even

# If odd, a0 = 3a0+1
Odd:
slli t0 a0 1
add a0 a0 t0
addi a0 a0 1
j Finally # Skip past Even code

# If even, a0 = a0/2
Even:
srli a0 a0 1

# Jump back to start of loop
Finally:
j Loop
Done:
# Exit program
li a0 10
li a1 0
ecall
```

Hailstone: Where we left off

```
# Sets a0 to starting value of Hailstone Sequence
li a0 27

Loop:
mv s0 a0 # Save value of a0 in s0 so we don't lose it
```

```
# "Call" print function somehow
```

```
mv a0 s0 # Restore a0 from s0 after print
```

```
# If a0 == 1, we're done; exit program
li t0 1
beq t0 a0 Done
# Check if a0 is even or odd
andi t1 a0 1
beq t1 x0 Even
```

```
# If odd,  $a0 = 3a0 + 1$ 
Odd:
slli t0 a0 1
add a0 a0 t0
addi a0 a0 1
j Finally # Skip past Even code
```

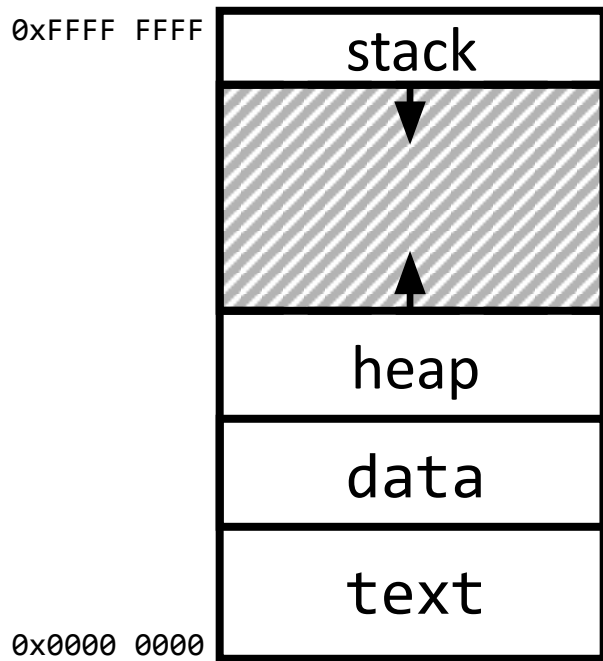
```
# If even,  $a0 = a0 / 2$ 
Even:
srli a0 a0 1
```

```
# Jump back to start of loop
Finally:
j Loop
Done:
# Exit program
li a0 10
li a1 0
ecall
```

```
print:
add a1 a0 x0
li a0 1
ecall
li a0 11
li a1 10
ecall
# return control to caller somehow
```

Review: C Memory Model

- In C, memory was divided into four segments:
 - Code/Text
 - Static/Data
 - Heap
 - Stack
- RISC-V uses the same memory layout.



Text

- RISC-V code is also a form of data. This data gets stored in the text section of memory.
- In RISC-V, every (real) instruction is stored as a 32-bit number.
 - The "next" instruction is always stored 4 bytes after the current instruction.
- A special 33rd register called the **Program Counter** (or PC) keeps track of which line of code is currently being run.



Current Line

Address	Data
0x0000 0000	li a0 27
0x0000 0004	mv s0 a0
0x0000 0008	add a1 a0 x0
0x0000 000C	li a0 1
0x0000 0010	ecall
0x0000 0014	li a0 11

Register	Value
PC	0x0000 0008

RISC-V Jump Instructions

- `jal rd Label`: Jump And Link
 - Jumps to the given label, but also sets `rd` to `PC+4` (the line after the current line)
 - Ex. if there was a `jal` on line `0x0000 0008`, `rd` would be set to `0x0000 000C`
- `jr rs1`: Jump Register
 - Jumps to the instruction at address `rs1`



Current Line

Address	Data
0x0000 0000	li a0 27
0x0000 0004	mv s0 a0
0x0000 0008	add a1 a0 x0
0x0000 000C	li a0 1
0x0000 0010	ecall
0x0000 0014	li a0 11

Register	Value
PC	0x0000 0008

Writing Hailstone with a print function

Modify the Hailstone code so that the print code is separated as a function:

- "Call" the function, saving a pointer to right after the function call (Return Address)
 - Use register `x1 = ra` to save the return address
- At the end of the print code, "return" to right after the function call

How are you doing on the question?

CS 61C

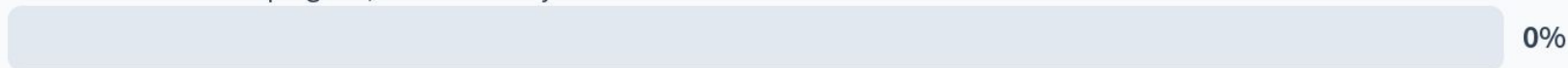
Summer 2026



I don't have a working solution yet, but have made good progress



I haven't made much progress, but have a way forward



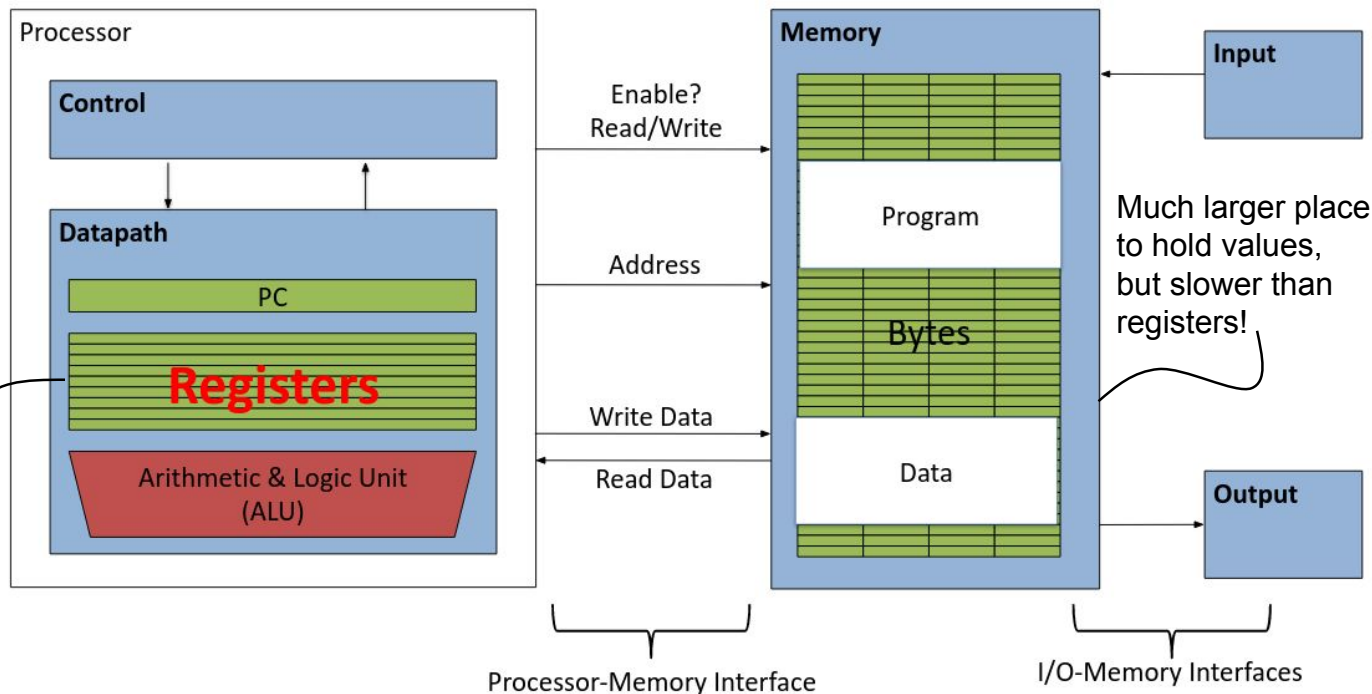
I don't know what's going on, and need more guidance before I can start



Agenda

- RISC-V Functions (Basic)
- **RISC-V Memory Operations**
- RISC-V Functions (Full)

Data Transfer: Load from and Store to memory



Very fast,
but limited space to hold values!

Review: How memory works

- On a 32-bit system, main memory contains 2^{32} bytes. Every 32-bit number acts as the **address** of one byte.
- 4 bytes together make a **word**. In order to store a word in memory, we cut the word up into 4 bytes, then store those bytes in consecutive addresses. In order to read a word, we read 4 consecutive addresses, then stitch those bytes back together
- RISC-V uses little-endian to store data: The least significant byte gets stored at the lowest address.
- By convention we say that a word is stored at its lowest address
 - Ex. The word stored at address 0x1000 is composed of the bytes 0x1000, 0x1001, 0x1002, and 0x1003

Load Word

Load Word syntax:

- `lw rd imm(rs1)`

Means: Compute $\text{imm} + \text{rs1}$, then load the 4 bytes at that address into `rd`

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Load Word

Example: `lw x10 12(x5)` if `x5` is `0x100`

- $0x100 + 12 = 0x10C$
- Bytes at `0x10C-0x10F` are `0x53`, `0x42`, `0x56`, `0x00`
- Since RISC-V is little-endian, this is the 32-bit value `0x0056 4253`
- So register `x10` will now store `0x0056 4253`

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Store Word

Store Word syntax:

- `sw rs2 imm(rs1)`

Means: Compute `imm+rs1`, then store the 4 bytes of `rs2` into that address.

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Store Word

Example: `sw x10 0(x5)` if `x5` is `0x100`, `x10` is `0x1234 5678`

- $0x100 + 0 = 0x100$
- Since RISC-V is little-endian, the 32-bit value `0x1234 5678` gets split into bytes `0x78 0x56 0x34 0x12`
- So the bytes in memory `0x100`, `0x101`, `0x102`, and `0x103` get set to `0x78`, `0x56`, `0x34`, and `0x12`, respectively.

Byte (0x)	78	56	34	12	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Loading and Storing Bytes

In addition to word data transfers (lw, sw), RISC-V has byte data transfers:

- `lb rd imm(rs1)`
- `sb rs2 imm(rs1)`

Load and store one byte instead of a full word

Problem: If registers contain 4 bytes, how do we load/store only 1 byte?

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Store Byte

Example: `sb x10, 0(x5)` if `x5` is `0x100`, `x10` is `0x1234 5678`

For `sb`, we store the *least significant byte*.

In the above example, `0x78` is the LSB.

So `x100` gets set to `0x78`

Byte (0x)	78	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Load Byte

Example: `lb x10, 0(x5)` if `x5` is `0x100`

For `lb`, we *extend* the numeric value to a full 32 bits.

In the above example, we load the number `0xEF`. What 32-bit number has the same numeric value as `0xEF`?

Answer: It depends on your representation scheme

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Loading and Storing Bytes

In addition to word data transfers (lw, sw), RISC-V has byte data transfers:

- `lb rd imm(rs1)` -> sign extend the byte
- `lbu rd imm(rs1)` -> zero extend the byte
- `sb rs2 imm(rs1)` -> store least significant byte only

Load and store one byte instead of a full word

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

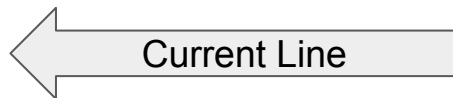
```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```

Example: What is in x12?

```
li x11, 0x93F5
```



```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```

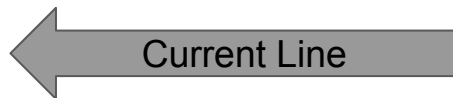
Register	Value
x11	0x00000000
x12	0x00000000
x5	0x00000100

Random
garbage since
data isn't set

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```



```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```

Register	Value
x11	0x000093F5
x12	0x00000000
x5	0x00000100

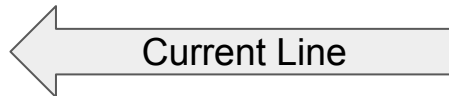
Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```



Register	Value
x11	0x000093F5
x12	0x00000000
x5	0x00000100

Byte (0x)	EF	BE	AD	DE	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```



Register	Value
x11	0x000093F5
x12	0x00000000
x5	0x00000100

Little-Endian
order

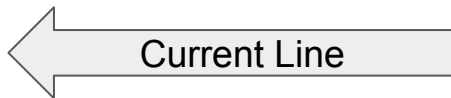
Byte (0x)	F5	93	00	00	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11, 0x93F5
```

```
sw x11, 0(x5)
```

```
lb x12, 1(x5)
```



Register	Value
x11	0x000093F5
x12	0x00000000
x5	0x00000100

Byte (0x)	F5	93	00	00	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

Example: What is in x12?

```
li x11,0x93F5
```

```
sw x11,0(x5)
```

```
lb x12,1(x5)
```



Register	Value
x11	0x000093F5
x12	0xFFFFFFFF93
x5	0x00000100

Sign-extend: Top
bit of 0x93 is 1,
so fill with 1s

Byte (0x)	F5	93	00	00	43	53	36	31	43	20	52	49	53	42	56	00
Address (0x)	100	101	102	103	104	105	106	107	108	109	10A	10B	10C	10D	10E	10F

x2: The Stack Pointer (sp)

The register x2 is set to a special value at the start of a program. Normally, it's used to keep track of the **stack**.

- Everything above sp when you start the program is **reserved**
- Everything below sp is **unallocated**
- Everything between sp and original location of sp is useable space for you.

To use the stack:

- Move sp down
- Store your data in the stack

Example Stack use

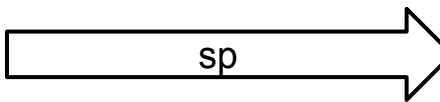
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

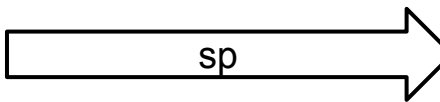
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

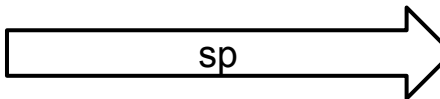
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFFC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

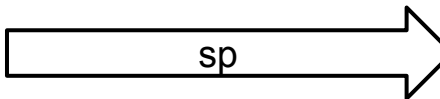
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

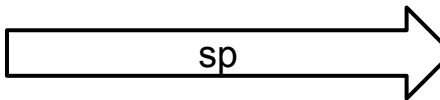
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0	EF	BE	AD	DE
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Example Stack use

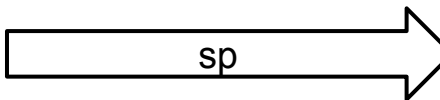
Assume sp starts at 0xFF4

```
addi sp sp -4
```

```
li t0 0xDEADBEEF
```

```
sw t0 0(sp)
```

```
addi sp sp 4
```



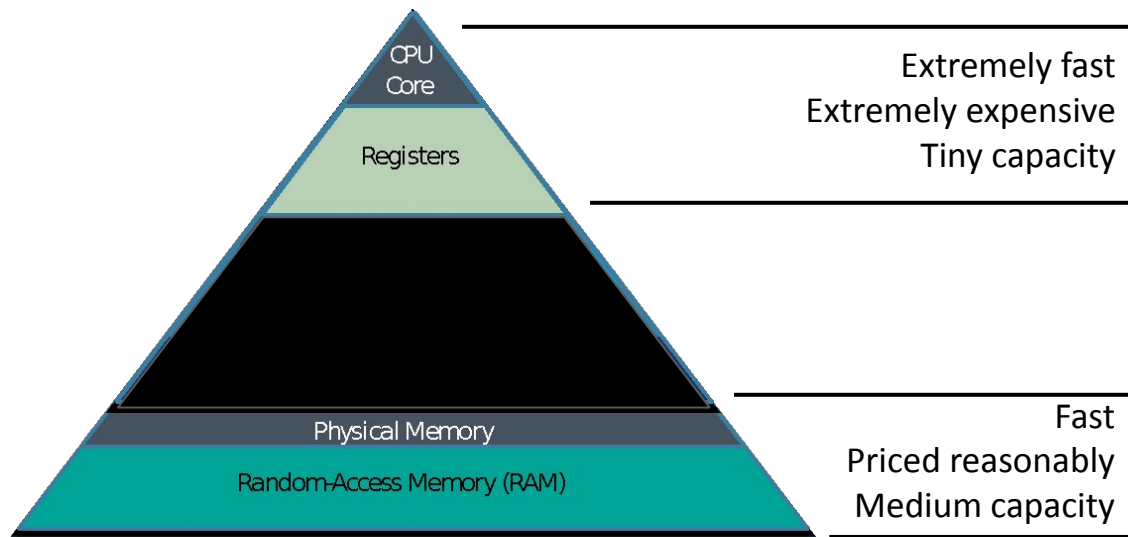
	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0	EF	BE	AD	DE
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Writing Reverse Hailstone

Write in Venus a program that starts with a value in `a0`, and prints out the hailstone sequence starting from that value **in reverse order**.

- You may use `t0` and `t1` as "free cells" for temporary values, and `s0` and `s1` for more permanent variables
- You may use the `sp` as defined previously (only space below where `sp` was when you start the program, and above where `sp` is now)

RISC-V Guiding Philosophy



Speed of Registers vs Memory

- Given that
 - Registers: 32 words (128 Bytes)
 - Memory (DRAM): Billions of bytes (2 GB to 96 GB on laptop)
- and physics dictates...
 - Smaller is faster
- How much faster are registers than DRAM??
 - About 50-500 times faster!
(in terms of latency of one access - tens of ns)
 - But subsequent words come every few ns

Jim Gray's Storage Latency Analogy: How Far Away is the Data?

CS 61C

100 Memory



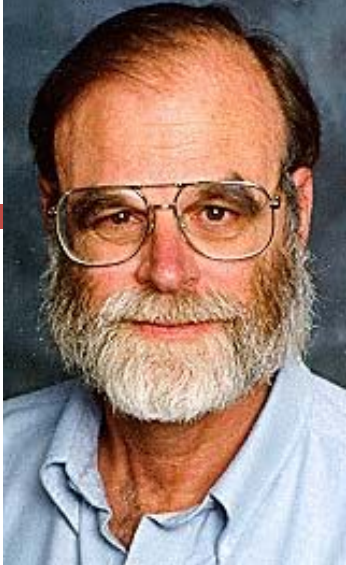
Sacramento

1 Registers



My Head

1 min



Jim Gray
Turing Award
B.S. Cal 1966
Ph.D. Cal
1969

1.5 hr

And in Conclusion...

- Memory is byte-addressable, but **lw** and **sw** access one word at a time.
- A pointer (used by **lw** and **sw**) is just a memory address, we can add to it or subtract from it (using offset).
- Memory can be used for variables we can't store in registers, but 100x slower than using registers directly
 - Use loads and stores as infrequently as possible!

Agenda

- RISC-V Functions (Basic)
- RISC-V Memory Operations
- **RISC-V Functions (Full)**

Print function

```
print:
add a1 a0 x0
li a0 1
ecall
li a0 11
li a1 10
ecall
jr ra
```

What does the user of this print function need to know in order to properly interface with it?

- Where to send input (a0)
- Where to send return address (ra)
- What registers get modified (a0, a1)

Need to define these rules for every function

Better: Establish a set of **conventions** that all functions follow

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

Register	Role

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

x0 = zero: Always zero

Register	Role

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

`x0` = zero: Always zero

Register	Role
zero	Zero

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

`x1` = `ra`: Return Address

A function assumes `ra` is where to jump back to when returning

Register	Role
zero	Zero
ra	Return Address

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

`x2 = sp`: Stack Pointer

Points to bottom of stack.

A function can allocate space on the stack for its own memory, but can't change anything above where `sp` was when the function was called.

After returning, `sp` should be back where it was originally.

Register	Role
zero	Zero
ra	Return Address
sp	Stack Pointer

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

$x3 = gp$, $x4 = tp$

Global Pointer and Thread Pointer. Out of scope, used for heap and multithreading.

Register	Role
zero	Zero
ra	Return Address
sp, gp, tp	Various Pointers

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

`x10-x17 = a0-a7:`

Argument Registers

Used to store function arguments.

`a0` and `a1` also used for return values.

Can be changed by functions (can't assume unchanged after function call)

Register	Role
zero	Zero
ra	Return Address
sp, gp, tp	Various Pointers
a0-a7	Argument

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

$x5-x7, x28-x31 = t0-t6$

Temporary Registers

Used for intermediate calculations

Can be changed by functions (can't assume unchanged after function call)

Register	Role
zero	Zero
ra	Return Address
sp, gp, tp	Various Pointers
a0-a7	Argument
t0-t6	Temporary

Calling Convention

In RISC-V, all functions follow the following conventions for register use:

`x8-x9, x18-x27 = s0-s11`

Saved Registers

Used for permanent variables

Can't be changed by functions

- If a function uses `s0`, it's responsible for restoring its old value before returning
- How? Use the stack!

Register	Role
zero	Zero
ra	Return Address
sp, gp, tp	Various Pointers
a0-a7	Argument
t0-t6	Temporary
s0-s11	Saved

Saving things on the stack

Assume sp starts at 0xFF4

```
addi sp sp -12
```

```
sw s0 0(sp)
```

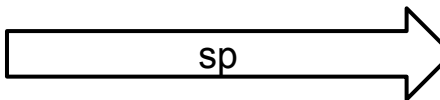
```
sw s1 4(sp)
```

...

```
lw s0 0(sp)
```

```
lw s1 4(sp)
```

```
addi sp sp 12
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Saving things on the stack

Assume sp starts at 0xFF4

```
addi sp sp -12
```

```
sw s0 0(sp)
```

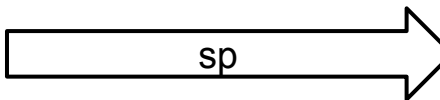
```
sw s1 4(sp)
```

...

```
lw s0 0(sp)
```

```
lw s1 4(sp)
```

```
addi sp sp 12
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Saving things on the stack

Assume sp starts at 0xFF4

```
addi sp sp -12
```

```
sw s0 0(sp)
```

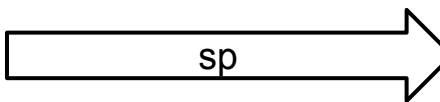
```
sw s1 4(sp)
```

...

```
lw s0 0(sp)
```

```
lw s1 4(sp)
```

```
addi sp sp 12
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC				
0xFE8				
0xFE4				
0xFE0				
...				

Saving things on the stack

Assume sp starts at 0xFF4

```
addi sp sp -12
```

```
sw s0 0(sp)
```

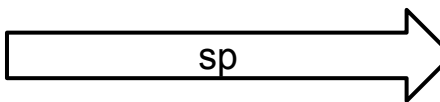
```
sw s1 4(sp)
```

...

```
lw s0 0(sp)
```

```
lw s1 4(sp)
```

```
addi sp sp 12
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC	old s1			
0xFE8	old s0			
0xFE4				
0xFE0				
...				

Saving things on the stack

Assume sp starts at 0xFF4

```
addi sp sp -12
```

```
sw s0 0(sp)
```

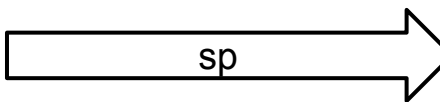
```
sw s1 4(sp)
```

...

```
lw s0 0(sp)
```

```
lw s1 4(sp)
```

```
addi sp sp 12
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC	old s1			
0xFE8	old s0			
0xFE4				
0xFE0				
...				

Saving things on the stack

Assume sp starts at 0xFF4

```
addi sp sp -12
```

```
sw s0 0(sp)
```

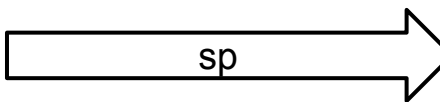
```
sw s1 4(sp)
```

...

```
lw s0 0(sp)
```

```
lw s1 4(sp)
```

```
addi sp sp 12
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC	old s1			
0xFE8	old s0			
0xFE4				
0xFE0				
...				

Saving things on the stack

Assume sp starts at 0xFF4

```
addi sp sp -12
```

```
sw s0 0(sp)
```

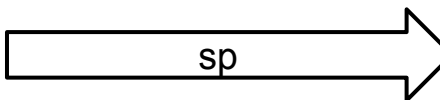
```
sw s1 4(sp)
```

...

```
lw s0 0(sp)
```

```
lw s1 4(sp)
```

```
addi sp sp 12
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0				
0xFEC	old s1			
0xFE8	old s0			
0xFE4				
0xFE0				
...				

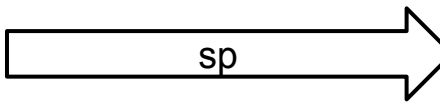
Saving things on the stack

If we call a function, need to save ra too!

```
addi sp sp -12
sw s0 0(sp)
sw s1 4(sp)
sw ra 8(sp)
```

...

```
lw s0 0(sp)
lw s1 4(sp)
lw ra 8(sp)
addi sp sp 12
```



	+0	+1	+2	+3
0xFFC				
0xFF8				
0xFF4				
0xFF0	saved ra			
0xFEC	old s1			
0xFE8	old s0			
0xFE4				
0xFE0				
...				

Saving things on the stack

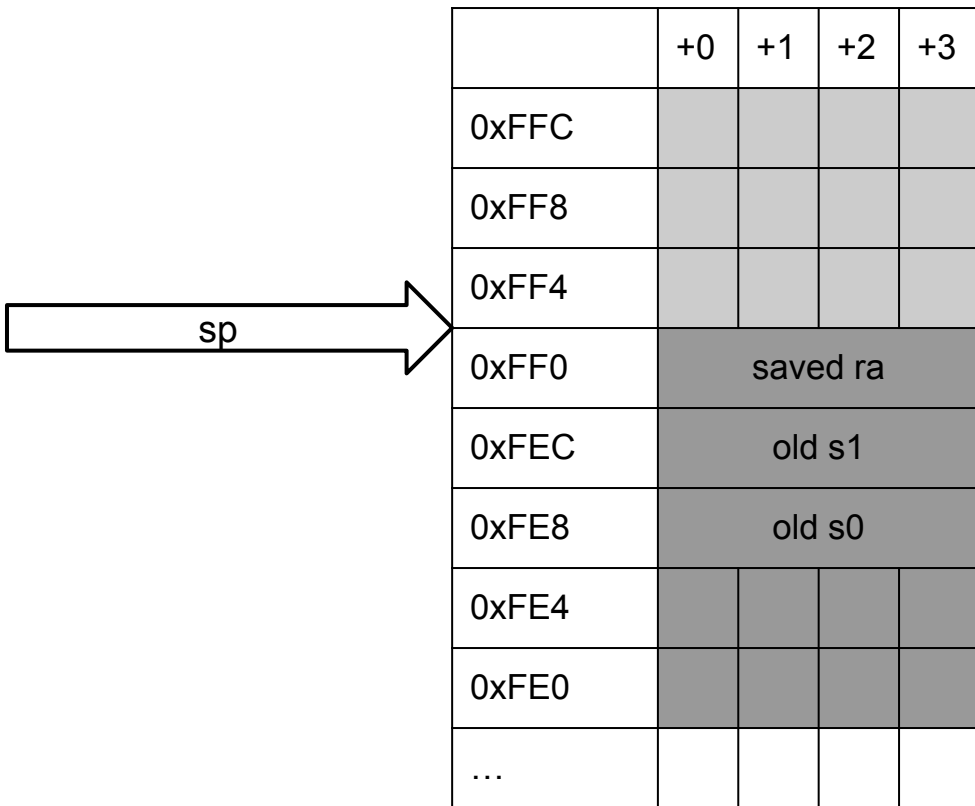
Prologue

```
addi sp sp -12
sw s0 0(sp)
sw s1 4(sp)
sw ra 8(sp)
```

...

Epilogue

```
lw s0 0(sp)
lw s1 4(sp)
sw ra 8(sp)
addi sp sp 12
```



Writing Hailstone as a function

Convert the Hailstone code into a function:

- Inputs: `a0 = N`, the starting point for the hailstone sequence
- Output: `a0`, return the largest value the sequence hits

Print each value of the hailstone sequence as before.

Instead of exiting when finished, return instead.

Remember to follow calling convention!

- Even though `print` doesn't affect many registers, you **must** assume that all `t` registers get modified, and you **must** restore any `s` registers you use.

Writing Hailstone as a function

```
li s0 27 # Test Code
li s1 871 # Insert this before your hailstone and print functions
mv a0 s0
jal ra hailstone # Call hailstone on 27
mv s0 a0
mv a0 s1
jal ra hailstone # Call hailstone on 871
mv s1 a0
mv a0 s0
jal ra print # Should print 9232
mv a0 s1
jal ra print # Should print 190996
li a0 10
li a1 0
ecall
```